Lecture 6
Chapter 5: Advanced SQL

Md. Tariquzzaman
Junior Lecturer, Dept. of Computer Science and Engineering
Islamic University of Technology (IUT)

CSE 4307: Database Management Systems

Chapter Outline

- Accessing SQL from Programming Languages
- Functions and Procedures
- Triggers
- Recursive Queries
- Advanced Aggregation Features (OLAP)

Accessing SQL from Programs

How to Connect DBMS with Real World Apps

The Challenge and Approaches

Why Interface with Programming Languages?

SQL is not general-purpose. Real-world apps (web servers, GUIs) need languages like Java/Python to handle logic and interfaces while SQL handles data.

Main Approaches:

- **Dynamic SQL**: Queries constructed as strings at runtime.
  - **JDBC (Java Database Connectivity)**: Standard API for Java.
  - **ODBC (Open Database Connectivity)**: Standard for C/C++ and others.
- **Embedded SQL**: SQL statements identified at compile time by a preprocessor (e.g., `EXEC SQL`).

Introduction to JDBC

JDBC Overview

Definition: JDBC is a Java API for communicating with database systems supporting SQL.

Key Capabilities:

- Querying and updating data.
- Retrieving query results.
- **Metadata retrieval**: Querying about relations present in the database, names, and types of relation attributes.

Communication Model:

1. Open a connection.
2. Create a ”statement” object.
3. Execute queries using the statement (send queries/fetch results).
4. Handle errors via the Exception mechanism.

Establishing Connections

JDBC Code (Modern Approach)

Works with Java 7 and JDBC 4 onwards.

```java
public static void JDBCexample(String dbid, String userid, String passwd) {
  try (Connection conn = DriverManager.getConnection(
  "jdbc:oracle:thin:@db.yale.edu:2000:univdb", userid, passwd);
  Statement stmt = conn.createStatement();
  ) {
    // ... Do Actual Work ...
  }
  catch (SQLException sqle) {
    System.out.println("SQLException : " + sqle);
  }
}
```

Note: Try-with-resources

Resources opened in the `try (...)` syntax are **automatically closed** at the end of the try block.

JDBC Code (Legacy Approach)

Older versions of Java/JDBC.

```
public static void JDBCexample(String dbid, String userid, String passwd) {
  try {
    Class.forName ("oracle.jdbc.driver.OracleDriver");
    Connection conn = DriverManager.getConnection(
    "jdbc:oracle:thin:@db.yale.edu:2000:univdb", userid, passwd);
    Statement stmt = conn.createStatement();
    // ... Do Actual Work ...
    stmt.close();
    conn.close();
  }
  catch (SQLException sqle) {
    System.out.println("SQLException : " + sqle);
  }
}
```

- `Class.forName` is **not required** from JDBC 4 onwards.
- Manual `close()` calls are required (error-prone).

Executing SQL Commands

Updates and Queries

1. Update Database

```
try {
  stmt.executeUpdate(
  "insert into instructor values('77987', 'Kim', 'Physics', 98000)");
} catch (SQLException sqle) {
  System.out.println("Could not insert tuple. " + sqle);
}
```

2. Execute Query & Fetch Results

```
ResultSet rset = stmt.executeQuery(
"select dept_name, avg (salary) from instructor group by dept_name");
while (rset.next()) {
  System.out.println(rset.getString("dept_name") + " " + rset.getFloat(2));
}
```

Result Details

Retrieving Fields

- `rs.getString("dept_name")` and `rs.getString(1)` are equivalent if `dept_name` is the first column in the result.

Dealing with Null Values

- Primitive types (int, float) cannot be null in Java.
- Use `wasNull()` to check the last read column.

```
int a = rs.getInt("a");
if (rs.wasNull()) {
  System.out.println("Got null value");
}
```

SQL Injection

Understanding the Attack

Scenario: A query constructed dynamically:

```
"select * from instructor where name = '" + name + "'"
```

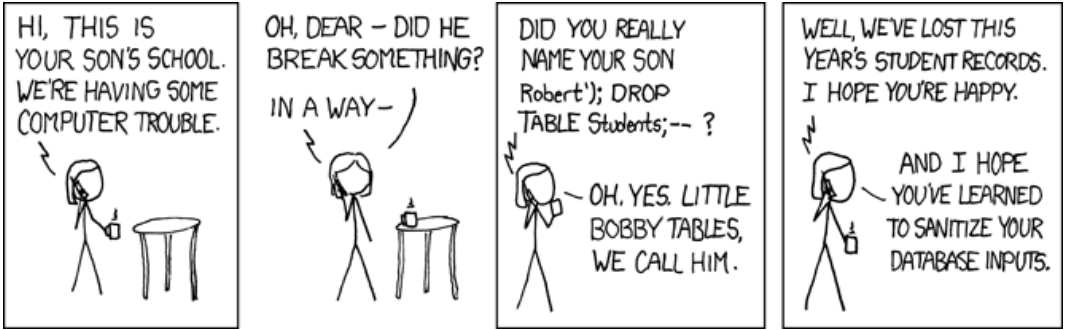
The Exploit: User enters: `X' or 'Y' = 'Y`

Resulting Query:

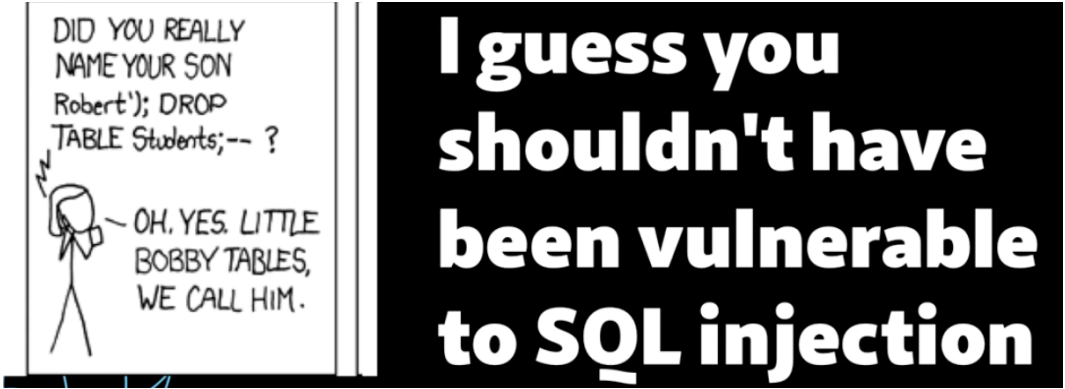
```
select * from instructor where name = 'X' or 'Y' = 'Y'
```

- The condition `'Y'='Y'` is always true.
- The attacker can access **all** data.

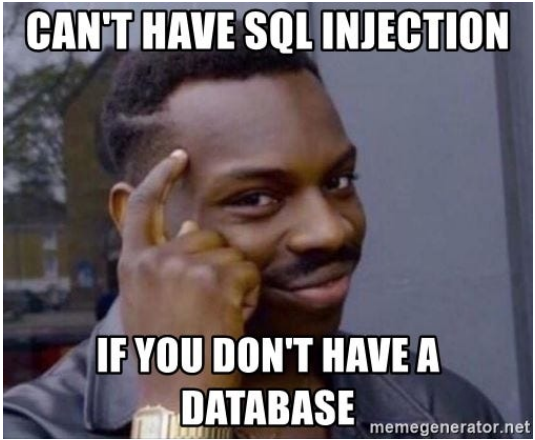
SQL Injection



SQL Injection



SQL Injection Solution



Prevention

Advanced Attacks: A user could even input: `X'; update instructor set salary = salary + 10000; --`

The Solution: Prepared Statements

- Prepared statements handle escaping automatically.
- Internally, it treats the input as a literal string: `select * from instructor where name = 'X\' or \'Y\' = \'Y'`
- **Rule**: Always use prepared statements with user inputs as parameters.

Prepared Statements

Prepared Statements

- **Definition**: A query compiled once with placeholders (e.g., `?` or `%s`) where values are inserted later.

1. Performance

- The database reuses the query execution plan.
- Reduces parsing overhead.

2. Security

- Prevents **SQL Injection** attacks.
- Input is treated as literal data, not executable code.
- *Example*: `'; drop table student;` -- becomes harmless text.

Prepared Statement Syntax

```
PreparedStatement pStmt = conn.prepareStatement(
"insert into instructor values(?,?,?,?)");

pStmt.setString(1, "88877");
pStmt.setString(2, "Perry");
pStmt.setString(3, "Finance");
pStmt.setInt(4, 125000);

pStmt.executeUpdate();

pStmt.setString(1, "88878");
pStmt.executeUpdate();
```

Warning: Security Best Practices

Crucial Warning

Always use prepared statements when taking input from the user. **NEVER** create a query by concatenating strings.

Bad Practice (String Concatenation): `"insert into instructor values('" + ID + "', '" + name + "', ..."`

Why?

- What if the name is `D'Souza`?
- The apostrophe breaks the SQL syntax.
- It opens the door to **SQL Injection**.

Metadata Features

ResultSet Metadata

Useful for generic code that processes query results without knowing schema in advance.

```
// After executing query to get a ResultSet rs:
ResultSetMetaData rsmd = rs.getMetaData();

for(int i = 1; i <= rsmd.getColumnCount(); i++) {
  System.out.println(rsmd.getColumnName(i));
  System.out.println(rsmd.getColumnTypeName(i));
}
```

Common Use Case: Building generic reporting tools.

Database Metadata (Columns)

Retrieving schema information about tables and columns.

```
DatabaseMetaData dbmd = conn.getMetaData();

// Args: Catalog, Schema, Table, Column
// "null" = all; "%" = wildcard (SQL LIKE)
ResultSet rs = dbmd.getColumns(null, "univdb", "department", "%");

while( rs.next()) {
  System.out.println(rs.getString("COLUMN_NAME"));
  System.out.println(rs.getString("TYPE_NAME"));
}
```

Database Metadata (Tables)

```java
DatabaseMetaData dbmd = conn.getMetaData();

// Args: Catalog, Schema, Table, Types[]
// The last attribute is an array of types to return (e.g., "TABLE", "VIEW")
ResultSet rs = dbmd.getTables(null, null, "%", new String[] {"TABLE"});

while( rs.next()) {
  System.out.println(rs.getString("TABLE_NAME"));
}
```

Finding Primary Keys

```java
DatabaseMetaData dmd = connection.getMetaData();

// Args: Catalog, Schema, Table
ResultSet rs = dmd.getPrimaryKeys(null, null, tableName);

while(rs.next()){
  // KEY_SEQ indicates position (important for composite keys)
  System.out.println(
  rs.getString("KEY_SEQ") + ": " +
  rs.getString("COLUMN_NAME")
  );
}
```

Transaction Control & Other Features

Transaction Control in JDBC

- **Default Behavior**: Auto-commit is **ON**. Each SQL statement is a separate transaction.
- **Problem**: Bad for groups of related updates (atomicity).

Manual Control:

1. Turn off auto-commit: `conn.setAutoCommit(false);`
2. Perform operations.
3. Commit or Rollback explicitly:
   - `conn.commit();`
   - `conn.rollback();`

Other JDBC Features

Callable Statements (/Procedures)

```
// Call a function
CallableStatement cStmt1 = conn.prepareCall("{? = call some_func(?)}");
// Call a procedure
CallableStatement cStmt2 = conn.prepareCall("{call some_proc(?,?)}");
```

Large Object Types (LOBs)

- `getBlob()` / `getClob()`: Return Blob/Clob objects.
- Access data via `getBytes()` or streams.
- **Update**: `blob.setBlob(index, inputStream)`.

SQLJ: Embedded SQL in Java

- **Problem with JDBC**: It is overly dynamic; SQL errors are not caught until runtime.
- **SQLJ Solution**: Static checking at compile time.

```
#sql iterator deptInfoIter (String dept_name, int avgSal);
deptInfoIter iter = null;

#sql iter = {
  select dept_name, avg(salary)
  from instructor group by dept_name
};

while (iter.next()) {
  String deptName = iter.dept_name();
  int avgSal = iter.avgSal();
  System.out.println(deptName + " " + avgSal);
}
iter.close();
```

Database Access from Python

The Standard: Python DB-API

While Java uses JDBC, Python uses the **DB-API** standard. Most drivers (e.g., `psycopg2` for Postgres, `sqlite3`, `mysql-connector`) follow this pattern.

Key Concepts

- **Connection**: Handles the session with the DB.
- **Cursor**: Equivalent to JDBC's `Statement`. Used to execute queries and iterate over results.

Typical Workflow

1. `connect()` to database.
2. Create a `cursor()`.
3. `execute()` SQL (use placeholders!).
4. `fetchone()` or `fetchall()`.

Database Access from Python (cont.)

```python
import sqlite3
# 1. Connect
conn = sqlite3.connect('univ.db')
# 2. Get Cursor
cursor = conn.cursor()
# 3. Execute (Safe Parameterization)
# Note: Use '?' or '%s' based on driver
dept = 'Music'
cursor.execute(
"SELECT name FROM instructor WHERE dept_name = ?",
(dept,)
)
# 4. Fetch Results
for row in cursor.fetchall():
print(row[0]) # Access by index
conn.close()
```

Database Access from Python (cont.)

```
import psycopg2
# Connect to server
conn = psycopg2.connect(
"dbname=univdb user=postgres password=secret"
)
cur = conn.cursor()
dept_name = 'Music'
# Use '%s' for placeholders
cur.execute(
"SELECT name FROM instructor WHERE dept = %s",
(dept_name,)
)
for row in cur.fetchall():
print(row)
conn.close()
```

ODBC (Open DataBase Connectivity)

ODBC Overview

Definition

ODBC is a standard API that allows application programs (GUIs, spreadsheets, data mining tools) to communicate with database servers.

Key Characteristics:

- **Independence**: Works independent of the specific DBMS (Oracle, SQL Server, etc.) or operating system.
- **Language Agnostic**: Defines a C language API, but bindings exist for Python, C++, C#, etc.

Core Workflow:

1. Open a connection with a database.
2. Send queries and updates.
3. Retrieve results.

ODBC vs. Other Standards

ODBC vs. JDBC

- **Language**: JDBC is strictly for **Java**. ODBC is a **C API** used by C++, C#, and native apps.
- **Platform**: JDBC runs on the JVM (Write Once, Run Anywhere). ODBC relies on native OS drivers (DLLs on Windows, .so on Linux).

ODBC vs. Embedded SQL

- **Mechanism**:
  - *Embedded*: Uses preprocessor directives (e.g., `EXEC SQL`) mixed with code.
  - *ODBC*: Uses a **Call-Level Interface (CLI)**. It's just a library of functions invoked at runtime.
- **Compilation**: ODBC does not require a special pre-compiler; Embedded SQL does.

Embedded SQL

Introduction to Embedded SQL

- **Concept**: Embedding SQL queries directly into "Host Languages" like C, C++, Java, or Fortran.
- **Preprocessor**: A specialized preprocessor converts SQL statements into host-language API calls before compilation.

Basic Syntax

- Identified by `EXEC SQL` prefix.
- Ends with a semicolon (C/C++) or `END-EXEC` (COBOL).

Generic/C Style: `EXEC SQL <statement>;`

Java (SQLJ) Style: `#SQL { ... };`

Connections and Variables

1. **Connecting** Before executing commands, the program must connect:

```
EXEC SQL connect to server user user_name using password;
```

2. **Host Variables**

- Variables from the host language used inside SQL must be preceded by a **colon** (:).
- Must be declared inside a specific block.

```
EXEC SQL BEGIN DECLARE SECTION;
int credit_amount;
EXEC SQL END DECLARE SECTION;

// Usage in SQL:   ... WHERE tot_cred > :credit_amount
```

The Concept of a Cursor

The Impedance Mismatch Problem

SQL operates on **sets** of records (relations), while host languages (C, Java) operate on **single variables** or objects at a time.

What is a Cursor? A **cursor** is a mechanism that allows a program to step through the result set of a query one row at a time, bridging the gap between the set-oriented database and the record-oriented application.

Analogy:

- **SQL Result**: A book containing many pages (rows).
- **Host Program**: A reader who can only read one page at a time.
- **Cursor**: The bookmark indicating which page is currently being read.

Cursor Declaration

Queries returning multiple rows require a **Cursor**.

The Workflow

1. **Declare** $\rightarrow$ 2. **Open** $\rightarrow$ 3. **Fetch** $\rightarrow$ 4. **Close**

Step 1: Declare Cursor

```
EXEC SQL
declare c cursor for
select ID, name
from student
where tot_cred > :credit_amount;
```

Note: No query is executed at this stage. It just defines the structure.

Cursor Execution Cycle

Step 2: Open

- Executes the query using current variable values.
- Stores results in a temporary relation.

```
EXEC SQL open c;
```

Step 3: Fetch

- Retrieves one tuple into host variables.

```
EXEC SQL fetch c into :si, :sn;
```

Step 4: Check & Close

- **SQLSTATE**: A variable (SQLCA) is set to `'02000'` when no data is left.
- **Close**: Deletes the temporary relation.

```
EXEC SQL close c;
```

Updates with Embedded SQL

To modify data while iterating, the cursor must be declared **for update**.

```
-- 1. Declare for Update
EXEC SQL
declare c cursor for
select * from instructor where dept_name = 'Music'
for update;

-- 2. Iterate and Update "Current" Row
/* ... inside loop after fetch ... */
update instructor
set salary = salary + 1000
where current of c;
```

- `where current of c` targets the specific tuple currently held by the cursor.

Introduction to PL/pgSQL

What is PL/pgSQL?

Definition

PL/pgSQL (Procedural Language/PostgreSQL) extends standard SQL by adding procedural features found in programming languages.

SQL vs. PL/pgSQL:

- **SQL (Declarative)**: You tell the DB *what* you want (e.g., `SELECT * FROM table`).
- **PL/pgSQL (Procedural)**: You tell the DB *how* to do it using logic.

Key Features:

- **Variables**: Store data for later use.
- **Control Structures**: `IF-ELSE`, `LOOPS`.
- **Error Handling**: `TRY-CATCH` blocks.

The Anatomy of a Block

Every PL/pgSQL function or anonymous command follows this structure:

```
DO $$
DECLARE
-- 1. Variables defined here
counter integer := 0;
BEGIN
-- 2. Execution Logic here
counter := counter + 1;

-- 3. Output
RAISE NOTICE 'Count: %', counter;
END $$;
```

Key Parts:

1. `DO $$ ... $$`: The container. `$$` acts as quotes so you don't have to escape text.
2. `DECLARE`: Where you define types (Integer, Text, etc.).
3. `BEGIN ... END`: The transaction body.

Analyzing the Example

The goal: Count rows in the 'film' table and print the result.

```
DO $$
DECLARE
film_count integer;          -- 1. Define variable
BEGIN
-- 2. SQL to Variable Assignment
select count(*) into film_count
from film;

-- 3. Print to Console
raise notice 'The number of films: %', film_count;
END $$;
```

Crucial Concepts:

- `SELECT ... INTO`: This is how you move data from a table into a variable.
- `%`: A placeholder (like `%s` in C/Python) replaced by `film_count`.

Conditional Logic (IF/ELSE)

PL/pgSQL allows branching logic based on data values.

```
DO $$
DECLARE
len integer := 120;
BEGIN
IF len < 60 THEN
RAISE NOTICE 'Short Film';
ELSIF len >= 60 AND len < 120 THEN
RAISE NOTICE 'Standard Length';
ELSE
RAISE NOTICE 'Epic Length';
END IF;
END $$;
```

Loops (FOR LOOP)

You can iterate through ranges or query results.

```
DO $$
DECLARE
cnt integer;
BEGIN
-- Standard Range Loop
RAISE NOTICE 'Counting Up:';

FOR cnt IN 1..3 LOOP
RAISE NOTICE 'Number: %', cnt;
END LOOP;

RAISE NOTICE 'Done!';
END $$;
```

Output: Number: 1, Number: 2, Number: 3, Done!

Functions and Procedures

Introduction to PL/pgSQL

Concept

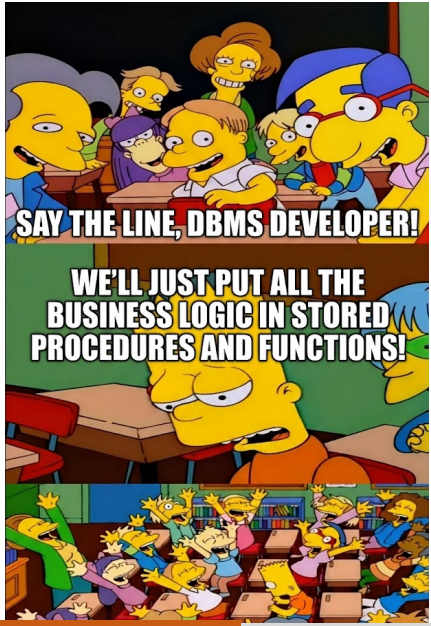
PostgreSQL uses **PL/pgSQL** (Procedural Language/PostgreSQL) to allow "business logic" to be stored in the database.

Key Differences form Standard SQL:

- **Block Structure**: Uses `DO $$ ... $$` or `CREATE FUNCTION ... AS $$ ... $$`.
- **Variables**: Must be declared in a `DECLARE` block.
- **Procedural Logic**: Supports loops (`LOOP`, `WHILE`, `FOR`) and conditionals.

Note: Unlike standard SQL functions, PL/pgSQL functions are defined as strings (using dollar quotes `$$`) that the database compiles.

Functions and Procedures



Declaring SQL Functions (Postgres Syntax)

Example: A function returning the count of instructors in a department.

```
CREATE OR REPLACE FUNCTION dept_count(d_name
   varchar)
RETURNS integer AS $$
DECLARE
d_count integer;
BEGIN
SELECT count(*) INTO d_count
FROM instructor
WHERE dept_name = d_name;

RETURN d_count;
END;
$$ LANGUAGE plpgsql;
```

Usage in Queries Can be used in SELECT and WHERE clauses:

```
SELECT dept_name, budget
FROM department
WHERE dept_count(dept_name) > 12;
```

Table Functions (Returning Sets)

Definition: Functions that return a table (set of rows) rather than a scalar value. Use `RETURNS TABLE`.

```
CREATE FUNCTION instructor_of(d_name char(20))
RETURNS TABLE (
ID varchar(5),
name varchar(20),
salary numeric(8,2)
) AS $$
BEGIN
-- RETURN QUERY streams results back
RETURN QUERY
SELECT i.ID, i.name, i.salary
FROM instructor i
WHERE i.dept_name = d_name;
END;
$$ LANGUAGE plpgsql;
```

Usage: Treated like a relation in the `FROM` clause.

```
SELECT * FROM instructor_of('Music');
```

Procedures vs. Functions in Postgres

Prior to Postgres 11, everything was a function. Now we have **Procedures**.

Functions

- invoked via `SELECT`.
- Must return a value (or void).
- **Cannot** manage transactions (cannot call `COMMIT` or `ROLLBACK`).

Procedures

- Invoked via `CALL`.
- No return value (uses `INOUT` parameters).
- **Can** manage transactions (can `COMMIT` inside the block).

Procedure Syntax

Scenario: Update salaries and commit the change immediately.

```
CREATE OR REPLACE PROCEDURE update_salaries(dept varchar, amount numeric)
LANGUAGE plpgsql
AS $$
BEGIN
UPDATE instructor
SET salary = salary + amount
WHERE dept_name = dept;

-- Transaction Control is allowed here!
COMMIT;
END;
$$;
```

Invoking a Procedure:

```
CALL update_salaries('Physics', 1000);
```

Transaction in Procedures

```
CREATE PROCEDURE massive_update()
LANGUAGE plpgsql
AS $$
BEGIN
-- Do some work
UPDATE accounts SET balance = balance * 1.05 WHERE type = 'Savings';

-- SAVE IT NOW. Even if the code crashes later, this update stays.
COMMIT;

-- Do more work
UPDATE accounts SET balance = balance - 10 WHERE type = 'Checking';
END;
$$;
```

Language Constructs: Loops

Postgres uses `FOR ... IN ... LOOP` to iterate over query results.

Example: Iterating over query results to calculate a sum manually.

```
DO $$
DECLARE
r RECORD; -- Special type to hold a row
total numeric := 0;
BEGIN
-- Loop through query results
FOR r IN SELECT budget FROM department WHERE dept_name = 'Music'
LOOP
total := total + r.budget;
END LOOP;

RAISE NOTICE 'Total Budget: %', total;
END $$;
```

External Language Routines

Why use External Languages (C/Python/Java)?

- **Efficiency**: C functions are compiled and loaded dynamically.
- **Capability**: Can perform system calls or use complex libraries (e.g., NumPy in PL/Python).

```sql
-- Example: Defining a C function in Postgres
-- Requires the compiled shared object (.so) file
CREATE FUNCTION add_one(integer)
RETURNS integer
AS '/usr/lib/postgresql/14/lib/my_funcs.so', 'add_one'
LANGUAGE C STRICT;
```

Security with External Routines

Allowing external code poses a risk to the database server.

Mitigation Strategies:

1. **Trusted vs. Untrusted Languages**:
   - `TRUSTED` (e.g., `plperl`, `plpgsql`): Sandboxed. Cannot access file system.
   - `UNTRUSTED` (e.g., `plpython3u`, `C`): Can access OS. Only superusers can create these.
2. **Process Isolation**: External code runs within the database process. If a C function segfaults, it can crash the entire database server.

Triggers

Trigger Fundamentals

Definition

A **trigger** is a procedural code block that is automatically executed ("fired") by the DBMS in response to specific changes in the database.

The ECA Model (Event-Condition-Action):

1. **Event**: The modification command that activates the trigger.
   - **Type**: `INSERT`, `UPDATE`, `DELETE`.
   - **Timing**: `BEFORE` (validate data) or `AFTER` (propagate changes).
2. **Condition**: A logical predicate (e.g., `NEW.grade IS NOT NULL`). If false, the action is skipped.
3. **Action**: The SQL statements to execute (can include updates to other tables).

Scenario: The Problem

Goal: Automatically maintain the `tot_cred` (total credits) for a student whenever they pass a course.

Input Tables (Initial State):

Table: Student

ID	Name	Tot_Cred
101	Alice	**0**

Table: Takes

ID	Course	Grade
101	CS-101	**NULL**

*(Note: CS-101 is worth **3** credits)*

The Solution: Trigger Code (Postgres)

In Postgres, you must first define a function, then bind it to a trigger.

```
-- 1. Create the Function
CREATE FUNCTION add_credits() RETURNS TRIGGER AS $$
BEGIN
-- Condition: Check if grade changed from fail/null to pass
IF NEW.grade <> 'F' AND NEW.grade IS NOT NULL
AND (OLD.grade = 'F' OR OLD.grade IS NULL) THEN
UPDATE student
SET tot_cred = tot_cred + (
SELECT credits FROM course WHERE course_id = NEW.course_id)
WHERE id = NEW.id;
END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;
-- 2. Create the Trigger
CREATE TRIGGER credits_earned
AFTER UPDATE OF grade ON takes
FOR EACH ROW
EXECUTE FUNCTION add_credits();
```

Execution Trace: How it Works

User Action: The instructor updates Alice's grade. `UPDATE takes SET grade = 'B' WHERE ID = '101';`

Trigger Logic Step-by-Step:

1. **Capture State**:
   - `OLD.grade` = NULL
   - `NEW.grade` = 'B'
2. **Evaluate Condition**: Is 'B' $\neq$ 'F'? **Yes**. Was old grade NULL/F? **Yes**. $\rightarrow$ **Proceed**.
3. **Execute Action**: Look up credits for CS-101 (3). Add 3 to Student 101.

Final Output (Table: Student):

ID	Name	Tot_Cred
101	Alice	**3**

Row-Level vs. Statement-Level Triggers

Row-Level Triggers

- Syntax: `FOR EACH ROW`
- Fires **once per tuple** modified.
- **Use Case**: Validating specific data values (e.g., "Salary cannot decrease").
- **Overhead**: High if updating 10,000 rows.

Statement-Level Triggers

- Syntax: `FOR EACH STATEMENT`
- Fires **once per transaction**.
- **Use Case**: Logging ("Table X was updated at 5 PM").
- **Access**: Uses **Transition Tables** (`old table`, `new table`) to see all affected rows at once.

Scenario: The ”Bulk Update” Test

To understand the difference, imagine we run a single SQL command that updates **2 rows** at once.

The Command:

```sql
UPDATE instructor
SET salary = salary + 1000
WHERE dept_name = 'Music';
```

(Assume the ’Music’ department has 2 instructors: Mozart and Beethoven.)

1. Row-Level Trigger Example

Goal: Log *every individual* salary change.

```
-- 1. The Function
CREATE FUNCTION log_row() RETURNS TRIGGER AS $$
BEGIN
INSERT INTO audit_log (msg)
VALUES ('Changed salary for ID: ' || NEW.ID);
RETURN NEW;
END; $$ LANGUAGE plpgsql;

-- 2. The Trigger
CREATE TRIGGER log_salary_change
AFTER UPDATE ON instructor
FOR EACH ROW EXECUTE FUNCTION log_row();
```

1. Row-Level Trigger Example

Execution Output (Audit Log Table):

Log_ID	Message
1	Changed salary for ID: 101
2	Changed salary for ID: 102

Result: The trigger fired **2 times** (Once per row).

2. Statement-Level Trigger Example

Goal: Log that a *transaction* occurred, without listing every ID.

```
-- 1. The Function
CREATE FUNCTION log_stmt() RETURNS TRIGGER AS $$
BEGIN
INSERT INTO audit_log (msg)
VALUES ('Batch update executed at ' || NOW());
RETURN NULL; -- Statement triggers ignore return values
END; $$ LANGUAGE plpgsql;

-- 2. The Trigger
CREATE TRIGGER log_salary_batch
AFTER UPDATE ON instructor
FOR EACH STATEMENT EXECUTE FUNCTION log_stmt();
```

2. Statement-Level Trigger Example

Execution Output (Audit Log Table):

Log_ID	Message
1	Salary batch update executed at 12:00:00

Result: The trigger fired **1 time** (Once per command), regardless of how many rows changed.

Comparison Summary

Feature	Row-Level	Statement-Level
Syntax	`FOR EACH ROW`	`FOR EACH STATEMENT`
Frequency	N times (N = rows affected)	1 time (per SQL command)
Var. Access	Can see `NEW.id`, `OLD.sal`	No direct row access
Best For...	Data Validation, Calc.	Auditing, Maintenance

The Risks of Triggers

The "Cascading" Danger

Trigger A updates Table B $\rightarrow$ Trigger B updates Table C $\rightarrow$ Trigger C updates Table A...

- Result: **Infinite Loop** or Stack Overflow.

Operational Risks:

- **Hidden Failures**: A minor error in an auditing trigger can roll back a critical financial transaction.
- **Bulk Operations**: Loading 1 million rows from a backup? Triggers will fire 1 million times, slowing the process to a crawl.
- *Best Practice: Disable triggers during bulk loads.*

Recursive Queries

Recursion in SQL

Concept

SQL:1999 permits recursive view definitions. This is essential for querying hierarchical data (e.g., org charts, prerequisite chains) where the depth is unknown.

Structure of a Recursive Query:

1. **Base Case**: Non-recursive query (the starting point).
2. **Recursive Step**: A query that joins the view with itself.

The WITH Clause (Common Table Expressions)

Definition

The WITH clause defines a temporary result set (a **Common Table Expression** or CTE) that exists only for the duration of a single SQL statement.

Why use it?

- **Readability**: Breaks complex queries into smaller, logical building blocks.
- **Simplification**: Avoids nested subqueries.

Example: Find departments with budget > average.

```
with avg_budget(value) as (
select avg(budget) from department
)
select dept_name, budget
from department, avg_budget
where department.budget > avg_budget.value;
```

Example: Transitive Closure

Goal: Find all courses that are prerequisites (directly or indirectly) for a specific course.

```
with recursive rec_prereq(course_id, prereq_id) as (
-- 1. Base Case: Direct prerequisites
select course_id, prereq_id
from prereq
union
-- 2. Recursive Step: Prereqs of prereqs
select rec_prereq.course_id, prereq.prereq_id
from rec_prereq, prereq
where rec_prereq.prereq_id = prereq.course_id
)
select * from rec_prereq;
```

This view computes the **transitive closure** of the ‘prereq‘ relation.

Execution Trace: How Recursion Works

The database computes the recursive view by iterating until no new rows are generated (Fixed Point).

Step 1: Initialization (S_0)

- Executes the **Base Case** query.
- Finds direct prerequisites (Depth 1).
- *Example*: (CSE-301 $\rightarrow$ CSE-201)

Step 2: Iteration 1 (S_1)

- Joins previous result (S_0) with `prereq`.
- Logic: If CSE-301 needs CSE-201, and CSE-201 needs CSE-101...
- New Tuple: (CSE-301 $\rightarrow$ CSE-101)

Step 3: Iteration N

- Uses S_1 to find Depth 3 dependencies.
- This continues repeatedly.

Step 4: Termination

- The process stops when an iteration produces **zero new rows**.
- The final result is the UNION of all steps ($S_0 \cup S_1 \cup \ldots \cup S_n$).

The Power of Recursion

Why not just use Joins?

Without recursion, you can only join the ‘prereq‘ table a fixed number of times.

Limitations of Standard SQL:

- A fixed query can only find managers/prerequisites up to a fixed depth (e.g., 5 levels).
- If the data has 6 levels, the query fails to retrieve all data.

The Recursive Solution:

- Iterates until a **Fixed Point** is reached (no new tuples are added).
- Handles hierarchies of arbitrary depth.

Advanced Aggregation (Ranking)

Ranking Functions

Ranking assigns a position to each tuple based on a value.

```
select ID, rank() over (order by GPA desc) as s_rank
from student_grades
order by s_rank
```

rank()

- Leaves gaps for ties.
- (1, 1, 3, 4)

dense_rank()

- No gaps.
- (1, 1, 2, 3)

Ranking within Partitions

We can reset the rank for each group (e.g., Rank students *within* their department).

```
select ID, dept_name,
rank() over (partition by dept_name order by GPA desc)
as dept_rank
from dept_grades
order by dept_name, dept_rank;
```

Benefits over LIMIT n:

- Allows finding "Top N" results **per category** in a single query.
- Standard SQL `LIMIT` applies to the entire result set, not subgroups.

Other Ranking Functions

- **percent_rank()**: Returns rank as a fraction (0 to 1).
- **cume_dist()**: Cumulative distribution.
- **row_number()**: Unique integer for every row (non-deterministic if ties exist).
- **ntile(n)**: Divides data into n buckets.

```sql
-- Divide students into 4 quartiles based on GPA
select ID, ntile(4) over (order by GPA desc) as quartile
from student_grades;
```

Windowing

Window Functions vs. Aggregation

Standard Aggregation

- `GROUP BY` collapses multiple rows into a single summary row.
- You lose the individual row details.

Result: Fewer rows than input.

Window Functions

- Calculates a value for **each row** based on a "window" of related rows.
- Does **not** collapse rows.

Result: Same number of rows as input.

Example 1: Moving Average

Goal: Smooth out sales data by averaging yesterday, today, and tomorrow.

```
select date, val,
avg(val) over (
order by date
rows between 1 preceding and 1 following
) as mov_avg
from sales
```

The "Window": For any given row, the calculation looks at the row before it, the current row, and the row after it.

Trace: Moving Average Calculation

Logic: $\frac{Prev+Curr+Next}{3}$

Date	Val	Window (Calculation)	Result
Jan 1	10	(Start + 10 + 20) / 2	**15.0**
Jan 2	20	(10 + 20 + 60) / 3	**30.0**
Jan 3	60	(20 + 60 + 10) / 3	**30.0**
Jan 4	10	(60 + 10 + End) / 2	**35.0**

Note how extreme values (like 60) are smoothed out in the result.

Example 2: Running Total (Partitioning)

Goal: Calculate a running bank balance, resetting for each customer.

```
select acct, date, amount,
sum(amount) over (
partition by acct
order by date
rows unbounded preceding
) as balance
from transactions
```

- **partition by acct**: Treat each account as a separate dataset.
- **rows unbounded preceding**: Sum from the very first record of this partition up to now.

Trace: Running Total Calculation

Acct	Date	Amt	Logic	Bal
A	Jan 1	+100	100	**100**
A	Jan 2	-20	100 - 20	**80**
A	Jan 3	-10	100 - 20 - 10	**70**
B	Jan 1	+500	*(Reset for new partition)*	**500**
B	Jan 5	+50	500 + 50	**550**

Window Frame Specifications

The `ROWS` clause defines exactly which rows are included in the window relative to the current row.

- **rows unbounded preceding**: From start of partition $\rightarrow$ Current Row. (Cumulative).
- **rows between 3 preceding and 3 following**: Rolling window of 7 rows (Current + 3 before + 3 after).
- **range between ...**: Operates on *values* rather than physical rows (e.g., "all transactions within the last 10 minutes", regardless of how many rows that is).

OLAP: Visualizing Multidimensional Data

The Scenario: Sales Data

To understand OLAP, imagine a standard Sales Table.

Item	Color	Size	Quantity
Shirt	Blue	L	10
Shirt	Blue	M	5
Pants	Black	L	20
Shirt	Red	L	12
...	...	...	...

- **Measure**: The number we want to sum/average (e.g., **Quantity**).
- **Dimensions**: The "filters" or categories (Item, Color, Size).

Step 1: The Cross-Tab (2D View)

Concept: A Cross-Tab (or Pivot Table) summarizes data on **two** dimensions simultaneously.

*Example: Sum of Quantity by **Item** (Rows) and **Color** (Columns).*

Item \Color	**Blue**	**Black**	**Red**	**Total**
Shirt	15	0	12	**27**
Pants	0	20	5	**25**
Total	**15**	**20**	**17**	**52**

Step 2: The Data Cube (3D View)

Problem: What if we want to analyze by Item, Color, **AND Size**?

The Data Cube Solution: Imagine the previous 2D table is just one "slice." A Data Cube stacks multiple 2D tables on top of each other (depth).

- **X-Axis**: Color
- **Y-Axis**: Item
- **Z-Axis**: Size (Small, Medium, Large)

A cell is defined by (Shirt, Blue, Large) $\rightarrow$ *10*

Operations: Slicing and Dicing

How do we extract information from this 3D Cube?

Slicing

- Fixing **one** dimension to a constant value.
- *Example*: "Show me sales for **Size = Large**."
- Result: A 2D Cross-Tab (Item vs. Color) for just Large items.

Dicing

- Selecting a specific **sub-cube** by filtering multiple dimensions.
- *Example*: "Show me sales for **Shirts** that are **Red or Blue**."

Operations: Hierarchy (Roll-up & Drill-down)

Dimensions often have a hierarchy (e.g., Time: Year $\rightarrow$ Quarter $\rightarrow$ Month).

- **Roll-up (Zoom Out)**: Moving from fine detail to a summary.
  - *Ex*: Summing "Daily Sales" to get "Monthly Sales".
  - *Ex*: Grouping "Cities" into "Countries".
- **Drill-down (Zoom In)**: Moving from a summary to fine detail.
  - *Ex*: Clicking on "Q1 Sales" to see the breakdown for Jan, Feb, Mar.

Extended Aggregation: Examples

Scenario: Input Data

Let's use this simple sales table to demonstrate the difference between `ROLLUP` and `CUBE`.

Table: Sales

Item	Color	Quantity
Shirt	Blue	10
Shirt	Red	5
Pants	Blue	20

Total Quantity = 35. Total Blue = 30. Total Shirt = 15.

1. The ROLLUP Output

Query: `GROUP BY ROLLUP(Item, Color)`

- Logic: Hierarchy (Item → Color → Total).
- **Missing**: It does **not** calculate totals for "Color" alone (e.g., Total Blue regardless of Item).

Item	Color	Sum	Description
Shirt	Blue	10	Specific Group
Shirt	Red	5	Specific Group
Shirt	**NULL**	**15**	*Subtotal: All Shirts*
Pants	Blue	20	Specific Group
Pants	Red	5	Specific Group
Pants	**NULL**	**25**	*Subtotal: All Pants*
NULL	**NULL**	**40**	*Grand Total*

2. The CUBE Output

Query: `GROUP BY CUBE(Item, Color)`

- Logic: All possible combinations (2^N).
- **Added Value**: It calculates what ROLLUP missed (Totals by Color).

Item	Color	Sum	Description
...	...	...	*(Includes all ROLLUP rows)*
NULL	**Blue**	**30**	**New: Total for Blue**
NULL	**Red**	**5**	**New: Total for Red**

CUBE generates the cross-sections necessary for "slicing" the data by any single dimension.

OLAP Operations & Implementation

User Operations:

- **Slicing/Dicing**: Fixing values for dimensions (e.g., Color='Red').
- **Roll-up**: Moving from fine to coarse granularity (Year $\leftarrow$ Month).
- **Drill-down**: Moving from coarse to fine granularity (Year $\rightarrow$ Month).

Implementation Strategies:

- **MOLAP** (Multidimensional): Arrays in memory. Fast, high space usage.
- **ROLAP** (Relational): Standard DB tables. Scalable, slower.
- **HOLAP** (Hybrid): Summaries in memory, base data on disk.

Questions?